



รายงานโครงงานวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology)

ภาคเรียนที่ 1 ปีการศึกษา 2569

โครงงานระบบจำลองห้องเล่นและวิเคราะห์เด็คการ์ดแบบเรียลไทม์บนคลาวด์ Serverless

(Disney Lorcana PlayLab Cloud: Real-Time Serverless Card Game Simulator)

กลุ่ม G21

รายงานความก้าวหน้าโครงการ (Stage 2: บทที่ 1 - 3 และส่วนขยายระบบคลาวด์)

คณะผู้จัดทำ:

- | | |
|----------------------------------|-----------------------|
| 1. นายชยุต บุญวัฒน์ | รหัสนักศึกษา 67070032 |
| 2. นายธนัทภัทร พรหมทอง | รหัสนักศึกษา 67070069 |
| 3. นายภูริ ประชาสุขสิน | รหัสนักศึกษา 67070137 |
| 4. นางสาววรรณนิศา อมรวงศ์ไพบุลย์ | รหัสนักศึกษา 67070155 |
| 5. นายอสิรารา พุ่มดอกไม้ | รหัสนักศึกษา 67070199 |
| 6. นายวรธิษฐ์ คงทอง | รหัสนักศึกษา 67070275 |

อาจารย์ประจำวิชา:

1. ดร. ชานนพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

กันยายน 2569

บทคัดย่อภาษาไทย

โครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** นำเสนอการวิเคราะห์ ออกแบบ และพัฒนาระบบจำลองห้องเล่นและวิเคราะห์
เด็คการ์ดแบบเรียลไทม์บนสถาปัตยกรรมคลาวด์ไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% AWS Serverless Architecture) เพื่อรองรับเกมการ์ด
สะสม Disney Lorcana Trading Card Game (TCG) โดยมุ่งเน้นการขจัดปัญหาภาระค่าใช้จ่ายเซิร์ฟเวอร์แบบเดิม (Server Overhead)
และลดความหน่วงในการส่งข้อมูลระหว่างผู้เล่น (Latency Reduction) ให้ต่ำกว่า 100 มิลลิวินาที (Sub-100ms)

โครงสร้างระบบขับเคลื่อนด้วยการประมวลผลตามเหตุการณ์ (Event-Driven Computing) ผ่าน AWS Lambda ในรูปแบบ
Microservices รองรับการสื่อสารแบบสองทิศทางความเร็วสูงผ่าน AWS API Gateway WebSockets ร่วมกับการจัดเก็บข้อมูลถาวรบน
Amazon DynamoDB ซึ่งปรับขนาดอัตโนมัติ (Auto-scaling) และการประมวลผลวิเคราะห์เด็คการ์ดแบบ Asynchronous ผ่าน Amazon
SQS ในส่วนของผู้ใช้งาน (Frontend) พัฒนาด้วย React 19, TypeScript 5, Tailwind CSS v4, Zustand State Machine และระบบ
ฟิสิกส์ 3D Card Inspector พร้อมเชื่อมต่อฐานข้อมูลการเทรดมาตรฐาน 408 ใบ

นอกจากนี้ รายงานฉบับนี้ได้ลงลึกถึงการออกแบบวิศวกรรมเพื่อความพร้อมใช้งานสูง (High Availability) และความเชื่อถือได้
(Reliability) การป้องกันความล้มเหลวจากการพึ่งพา API ภายนอกด้วยกลยุทธ์ Multi-tier Fallback และ Local Caching การจัดการสเกล
การเชื่อมต่อ WebSockets เมื่อมีห้องเล่นพร้อมกันจำนวนมากด้วย Partition Key Dispersion และ Connection Multiplexing ตลอดจน
การปฏิบัติตามกรอบ AWS Well-Architected Framework ภายใต้งบประมาณที่ควบคุมได้ (\$0.00 Free Tier และไม่เกิน \$50) รวมถึง
การแก้ไขข้อจำกัดเฉพาะของสภาพแวดล้อม AWS Learner Lab ได้อย่างสมบูรณ์แบบ

คำสำคัญ: คลาวด์ไร้เซิร์ฟเวอร์ (Serverless), เว็บซ็อกเก็ต (WebSockets), ดึงข้อมูลตามเหตุการณ์ (Event-Driven), เกมการ์ดดิจิทัล, AWS
Lambda, DynamoDB, API Gateway

Abstract

The **Disney Lorcana PlayLab Cloud (Group G21)** project presents the architectural design, engineering analysis, and implementation of a real-time multiplayer card simulator and asynchronous deck analytics platform built entirely on a 100% AWS Serverless Architecture for the Disney Lorcana Trading Card Game (TCG). The project targets the elimination of persistent infrastructure costs and achieves ultra-low synchronization latency (<100ms) between remote players.

The cloud backend is designed around event-driven microservices using AWS Lambda, bidirectional persistent connections via AWS API Gateway WebSockets, autoscaling document storage with Amazon DynamoDB, and decoupled asynchronous job processing through Amazon Simple Queue Service (SQS). The frontend client is engineered using React 19, TypeScript 5, Tailwind CSS v4, Zustand global state orchestration, Framer Motion, and CSS 3D matrix physics, delivering a high-fidelity tabletop simulation connected to an official catalog of 408 cards.

Furthermore, this Stage 2 report provides comprehensive engineering solutions for critical cloud challenges: mitigating external card API dependencies through multi-tier caching and local asset fallbacks, architecting WebSocket scalability under high concurrent room load using DynamoDB partition key dispersion, auto-reconnect grace windows, and adherence to the AWS Well-Architected Framework (Reliability, Availability, Security, and Cost under \$50 budget limit). Finally, it details the engineering workarounds engineered to overcome AWS Academy Learner Lab permission boundaries (pre-provisioned LabRole integration and automated CLI pipelines).

Keywords: AWS Serverless, WebSockets, Event-Driven Architecture, Digital TCG, AWS Lambda, DynamoDB, API Gateway, Cloud Reliability

กิตติกรรมประกาศ

โครงการ Disney Lorcana PlayLab Cloud สำเร็จลุล่วงตามวัตถุประสงค์ของการส่งมอบงานในระยะที่ 2 (Stage 2) ได้ด้วยความอนุเคราะห์ คำแนะนำ และการชี้แนะทางวิชาการอันทรงคุณค่ายิ่งจากคณาจารย์ประจำรายวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology) คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ได้แก่:

1. ดร. ธนเทพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศน์

ที่ได้กรุณาถ่ายทอดองค์ความรู้ด้านสถาปัตยกรรมคลาวด์ การคำนึงถึงความปลอดภัย ความพร้อมใช้งาน และการบริหารจัดการทรัพยากร อย่างคุ้มค่า ซึ่งเป็นรากฐานสำคัญในการออกแบบและแก้ปัญหาทางวิศวกรรมซอฟต์แวร์ในโครงการนี้

คณะผู้จัดทำขอขอบพระคุณเพื่อน ๆ นักศึกษาคณะเทคโนโลยีสารสนเทศทุกท่านที่ได้ร่วมทดสอบระบบกระดานและห้องเล่นแบบเรียลไทม์ ให้ข้อคิดเห็นด้านประสบการณ์การใช้งาน (UX/UI) และร่วมตรวจสอบข้อผิดพลาดของระบบ รวมถึงขอขอบคุณครอบครัวที่ให้การสนับสนุนและเป็นกำลังใจในการศึกษาค้นคว้าตลอดมา

คณะผู้จัดทำ โครงการกลุ่ม G21
กันยายน 2569

สารบัญ

บทคัดย่อภาษาไทย	i
Abstract	ii
กิตติกรรมประกาศ	iii
1 บทนำ (Introduction)	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของระบบ (System Scope)	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
1.5 โครงสร้างของรายงาน	2
2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Related Theories & Technologies)	4
2.1 สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์ (AWS Serverless Architecture)	4
2.2 สถาปัตย์กรรมการสื่อสารแบบสองทิศทางเรียลไทม์ (WebSockets Protocol)	4
2.3 เทคโนโลยีฝั่งผู้ใช้งาน (Frontend Engineering & State Architecture)	5
2.4 โมเดลโดเมนเกมการ์ดสะสม Disney Lorcana	5
2.5 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum	5
3 การวิเคราะห์และออกแบบระบบ (System Analysis & Architectural Design)	7
3.1 ข้อกำหนดความต้องการของระบบ (System Requirements)	7
3.1.1 ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements: FR)	7
3.1.2 ข้อกำหนดที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements: NFR)	7
3.2 สถาปัตยกรรมระบบโดยรวม (End-to-End Serverless Architecture)	8
3.3 การออกแบบ Use Case และลำดับการทำงาน (Use Case Design)	8
3.4 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)	9
3.5 การออกแบบการจัดการ API ภายนอกและการป้องกันความล้มเหลว (External API Dependency & High Availability Strategy)	9

3.5.1	การวิเคราะห์ความเสี่ยงเมื่อ API ภายนอกล้มเหลว	9
3.5.2	กลยุทธ์การแก้ไขปัญหาล้มเหลว 4 ระดับ (4-Tier Fallback Strategy)	9
3.6	การออกแบบการสเกลและรองรับการเชื่อมต่อ WebSockets พร้อมกัน (WebSocket Scalability & Concurrency Management)	10
3.6.1	ปัญหาคอขวดเมื่อมีผู้ใช้งานและห้องเล่นจำนวนมาก	10
3.6.2	แนวทางการแก้ไขปัญหาและการออกแบบเชิงสถาปัตยกรรม	10
3.7	การออกแบบตามกรอบ AWS Well-Architected Framework	11
3.7.1	ด้านความเชื่อถือได้ (Reliability)	11
3.7.2	ด้านความพร้อมใช้งาน (Availability)	11
3.7.3	ด้านความมั่นคงปลอดภัย (Security)	11
3.7.4	ด้านการบริหารจัดการต้นทุน (Cost Optimization under \$50 Budget)	12
3.8	การวิเคราะห์ข้อจำกัดของ AWS Learner Lab และแนวทางแก้ไขเชิงวิศวกรรม (Learner Lab Constraints & Engineering Workarounds)	12
4	การพัฒนาและผลการทดสอบคุณภาพ (Implementation & Quality Assurance)	14
4.1	รายละเอียดการพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation)	14
4.2	รายละเอียดการพัฒนาส่วนประมวลผลบนคลาวด์ (AWS Microservices)	15
4.3	แผนและผลการทดสอบคุณภาพระบบ (Quality Assurance & Test Verification)	15
5	สรุปผลการดำเนินงานและแผนงานในอนาคต (Conclusions & Future Roadmap)	17
5.1	สรุปผลการดำเนินงานความก้าวหน้า Stage 2	17
5.2	แผนงานการพัฒนาสำหรับ Stage 3 (Sprint 4 ถึง Sprint 7)	17
5.3	บทเรียนที่ได้รับและการปรับปรุงกระบวนการทำงาน (Retrospective)	17
	บรรณานุกรม (References)	19
	ภาคผนวก (Appendix)	20

สารบัญตาราง

2.1	แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint	6
3.1	ตารางสรุปรายละเอียด Use Cases ของระบบ (LC-01 ถึง LC-09)	8
3.2	โครงสร้างตารางข้อมูลบน Amazon DynamoDB	9
3.3	การประเมินค่าใช้จ่ายบนสถาปัตยกรรม AWS Serverless (ผู้ใช้งาน 10,000 คน/เดือน)	12
4.1	ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)	16

สารบัญรูปภาพ

3.1	สถาปัตยกรรมระบบคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% AWS Serverless)	8
4.1	ตัวอย่างภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud	14

บทที่ 1

บทนำ (Introduction)

1.1 ความเป็นมาและความสำคัญของปัญหา

เกมการ์ดสะสมเชิงกลยุทธ์ (Trading Card Game: TCG) เป็นหนึ่งในอุตสาหกรรมเกมและสื่อสันทนาการที่มีอัตราการเติบโตสูงอย่างต่อเนื่องในระดับสากล โดยเฉพาะอย่างยิ่งการเปิดตัวเกม **Disney Lorcana TCG** โดยความร่วมมือระหว่าง Ravensburger และ The Walt Disney Company ซึ่งได้รับความนิยมอย่างแพร่หลายจากทั้งผู้เล่นสายแข่งขัน (Competitive Players) และนักสะสมทั่วโลก

อย่างไรก็ดี ในบริบทของการฝึกซ้อมและการเข้าถึงเกมการ์ด ผู้เล่นส่วนใหญ่ยังคงประสบกับข้อจำกัดหลัก 3 ประการ:

1. **ต้นทุนการจัดหาการ์ดจริงและข้อจำกัดทางกายภาพ:** การ์ดหายากมีราคาสูงในตลาดรอง และการนัดหมายเล่นจำเป็นต้องใช้สถานที่จริง ทำให้ผู้เล่นใหม่ขาดโอกาสในการทดลองจัดเด็ค (Deck Building)
2. **ภาระต้นทุนเซิร์ฟเวอร์และความไร้ประสิทธิภาพของระบบดั้งเดิม:** ระบบจำลองเกมการ์ดบนอินเทอร์เน็ตในอดีตมักทำงานบน Virtual Machine หรือ Dedicated Server (เช่น Amazon EC2) ที่ต้องเปิดทิ้งไว้ตลอด 24 ชั่วโมง ก่อให้เกิดค่าใช้จ่ายคงที่สูงแม้ไม่มีผู้ใช้งาน และมักเกิดปัญหาเซิร์ฟเวอร์ล่มเมื่อมีปริมาณทราฟฟิกพุ่งสูงขึ้นอย่างกะทันหัน
3. **ความหน่วงในการส่งข้อมูล (High Latency):** แพลตฟอร์มบนเว็บทั่วไปมักใช้การดึงข้อมูลแบบ HTTP Polling หรือ Long Polling ซึ่งมีความหน่วงสูง ไม่ตอบสนองต่อการเล่นเกมการ์ดที่ต้องการการชิงชัยตำแหน่งการ์ด สถานะพร้อมใช้ (Ready/Exert) และแต้มคะแนน (Lore) แบบทันทีทันใด (Real-Time)

เพื่อแก้ไขปัญหาดังกล่าว คณะผู้จัดทำจึงได้ริเริ่มโครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** โดยนำสถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% AWS Serverless Architecture) ร่วมกับเทคโนโลยีเว็บสมัยใหม่ มาใช้ในการพัฒนาระบบจำลองกระดานเล่นการ์ดและระบบวิเคราะห์เด็คอัจฉริยะ ที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที สามารถขยายขนาดรองรับผู้เล่นได้โดยอัตโนมัติ และควบคุมค่าใช้จ่ายโครงสร้างพื้นฐานให้อยู่ภายใต้งบประมาณฟรีของ AWS Free Tier (\$0.00) ได้อย่างสมบูรณ์

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อออกแบบและพัฒนาระบบจำลองกระดานเล่นเกมการ์ด Disney Lorcana แบบเรียลไทม์บนเว็บเบราว์เซอร์ที่มีความหน่วงต่ำ (<100ms) ผ่านช่องทางสื่อสารสองทิศทาง WebSockets
2. เพื่อประยุกต์ใช้สถาปัตยกรรม AWS Serverless (AWS Lambda, Amazon DynamoDB, AWS API Gateway) ในการสร้างระบบ Microservices ที่ประหยัดพลังงาน ยืดหยุ่น และปรับขนาดตามโหลดอัตโนมัติ
3. เพื่อพัฒนาระบบจัดการเด็คการ์ด (Deck Builder) และระบบประมวลผลวิเคราะห์สมดุลเด็คแบบ Asynchronous ผ่าน Amazon SQS
4. เพื่อศึกษาและวางมาตรการด้านความเชื่อถือได้ (Reliability), ความพร้อมใช้งาน (Availability), ความปลอดภัย (Security) และการควบคุมต้นทุน (Cost Optimization) ตามกรอบ AWS Well-Architected Framework
5. เพื่อศึกษาและแก้ไขปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อมสิทธิ์จำกัดของ AWS Academy Learner Lab

1.3 ขอบเขตของระบบ (System Scope)

ระบบครอบคลุมฟังก์ชันการทำงานหลัก 9 ด้าน (9 Core Use Cases) ดังนี้:

1. **ระบบยืนยันตัวตนและความปลอดภัย (Authentication - LC-01, LC-02):** สมัครงานและเข้าสู่ระบบด้วยการเข้ารหัสผ่าน bcrypt (10 Salt Rounds) และออกใบรับรองความปลอดภัยด้วย JSON Web Token (JWT)
2. **ระบบจัดการดีคการ์ด (Deck Management - LC-03):** สร้าง ค้นหา แก้ไข และลบดีคการ์ดส่วนตัว บันทึกข้อมูลลงใน Amazon DynamoDB
3. **ระบบจำลองการเปิดซองการ์ด (Booster Pack Gacha Simulator - LC-04):** สุ่มเปิดการ์ด 12 ใบต่อซองตามสัดส่วนความน่าจะเป็นทางการ 9 ระดับ (Common ถึง Enchanted)
4. **ระบบตรวจสอบการ์ดแบบ 3 มิติ (3D Card Inspector - LC-05):** แสดงผลการ์ดแบบสามมิติ ตรวจสอบเอฟเฟกต์พอยล์และรายละเอียดข้อความของการ์ดอย่างคมชัด
5. **ระบบจำลองกระดานและห้องเล่นเรียลไทม์ (WebSockets Match Sync - LC-06):** จำลองกระดานเต็มรูปแบบ (Play Area, Inkwell, Hand, Discard, Deck, Lore Counter 0-20, Ready/Exert) และซิงค์การเคลื่อนไหวผ่าน WebSocket API Gateway
6. **ระบบวิเคราะห์ดีคการ์ดแบบ Asynchronous (Async Deck Analyzer - LC-07):** คำนวณค่าร้ายเฉลี่ย (Ink Curve) และวิเคราะห์ความเข้ากันได้ของการผ่านคิว Amazon SQS
7. **ระบบจัดการและดูแลผู้ใช้งาน (User Administration - LC-08):** จัดการระงับสิทธิ์บัญชีผู้ใช้เมื่อทำผิดกฎ
8. **ระบบตรวจสอบและติดตามการทำงาน (System Monitoring - LC-09):** ติดตามเมตริกการทำงาน ทราฟฟิก และ Error Logs ผ่าน Amazon CloudWatch

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้เล่นเกม Disney Lorcana มีเครื่องมือสำหรับฝึกซ้อม ทดสอบเด็ค และเล่นกับคู่แข่งผ่านเว็บเบราว์เซอร์ได้ทันทีโดยไม่ต้องติดตั้งซอฟต์แวร์
2. ได้รับองค์ความรู้และทักษะเชิงลึกในการออกแบบสถาปัตยกรรม Serverless Microservices บน Amazon Web Services ที่พร้อมรองรับการขยายตัวในระดับ Production
3. ได้แนวทางปฏิบัติเชิงวิศวกรรมที่เป็นรูปธรรมในการรับมือกับความล้มเหลวของ API ภายนอก และการจัดการสเกลการเชื่อมต่อ WebSockets พร้อมกันจำนวนมาก
4. สามารถลดภาระต้นทุนด้านโครงสร้างพื้นฐานคลาวด์ลงได้ 100% เมื่อเทียบกับการเช่าโฮสต์หรือ Virtual Machine แบบเดิม

1.5 โครงสร้างของรายงาน

รายงานโครงงานความก้าวหน้า Stage 2 ฉบับนี้แบ่งออกเป็น 5 บท ประกอบด้วย:

- **บทที่ 1 บทนำ:** นำเสนอความเป็นมา วัตถุประสงค์ ขอบเขต และประโยชน์ของโครงงาน

- **บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง:** อธิบายสถาปัตยกรรม Serverless, WebSockets, เทคโนโลยี Frontend, โมเดลเกม Lorcana และระเบียบวิธี Agile Scrum
- **บทที่ 3 การวิเคราะห์และออกแบบระบบ:** ครอบคลุมการออกแบบสถาปัตยกรรมคลาวด์ 4 เลเยอร์, Use Cases, DynamoDB Schema, การแก้ปัญหา API ภายนอก, การสเกล WebSockets, เสาหลัก Well-Architected และการเอาชนะข้อจำกัดของ AWS Learner Lab
- **บทที่ 4 การพัฒนาระบบและผลการทดสอบคุณภาพ:** รายละเอียดการพัฒนาส่วนติดต่อผู้ใช้, Backend Microservices, แผนและผลการทดสอบคุณภาพ (QA Test Matrix)
- **บทที่ 5 สรุปผลการดำเนินงานและแผนงานในอนาคต:** สรุปผลงาน Stage 2, แผนงาน Sprint 4-7 ใน Stage 3 และบทเรียนที่ได้รับ

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Related Theories & Technologies)

2.1 สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์ (AWS Serverless Architecture)

สถาปัตยกรรม Serverless เป็นรูปแบบการประมวลผลบนคลาวด์ที่ผู้พัฒนาไม่ต้องทำการจัดสรร ดูแล หรือบำรุงรักษาเครื่องเซิร์ฟเวอร์ (No Server Management) โดยคลาวด์โพรไวเดอร์เป็นผู้บริหารจัดการทรัพยากร การขยายขนาด (Auto-scaling) และความพร้อมใช้งานให้โดยอัตโนมัติ โดยคิดค่าบริการตามปริมาณการใช้งานจริงในระดับมิลลิวินาที (Pay-per-use)

โครงการนี้ได้ประยุกต์ใช้บริการคลาวด์หลักของ Amazon Web Services (AWS) ดังต่อไปนี้:

1. **AWS Lambda:** บริการประมวลผลโค้ดตามเหตุการณ์ (Function-as-a-Service: FaaS) ทำหน้าที่รัน Microservices ของระบบ ทั้งส่วน REST API และ WebSocket Event Routing โดยทำงานเฉพาะเมื่อมี Request เข้ามาและปิดตัวลงเมื่อทำงานเสร็จสิ้น
2. **Amazon API Gateway:** บริการจัดการ API แบบ Fully Managed ทำหน้าที่เป็นประตูทางเข้าของระบบ รองรับทั้ง HTTP API สำหรับการส่งคำขอแบบ Request-Response (REST) และ WebSocket API สำหรับการเชื่อมต่อแบบเปิดค้างไว้สองทิศทาง (Stateful Persistent Connection)
3. **Amazon DynamoDB:** ฐานข้อมูล NoSQL แบบ Key-Value และ Document Database ที่มีความเร็วสูงในระดับ Single-digit Millisecond และรองรับโหมดการคิดค่าบริการแบบตามปริมาณการใช้งานจริง (On-Demand / Pay-Per-Request)
4. **Amazon Simple Queue Service (SQS):** บริการคิวข้อความแบบกระจายตัวและไร้เซิร์ฟเวอร์ ทำหน้าที่เป็นตัวกลางในการส่งต่องานประมวลผลหนัก เช่น การคำนวณสถิติเพื่อลดภาระการรอคอยของผู้ใช้งาน (Decoupled Asynchronous Processing)
5. **Amazon CloudWatch:** บริการศูนย์รวมการเก็บ Logs เมตริกประสิทธิภาพ และการแจ้งเตือน (Alarms) เมื่อเกิดความผิดปกติในระบบ

2.2 สถาปัตยกรรมการสื่อสารแบบสองทิศทางเรียลไทม์ (WebSockets Protocol)

โพรโทคอล WebSocket (ตามมาตรฐาน RFC 6455) ทำงานบนช่องทาง TCP เดียวกันแบบ Full-Duplex ทำให้ Client และ Server สามารถส่งข้อมูลแลกเปลี่ยนกันได้ทันทีโดยไม่ต้องเริ่มต้น Handshake ใหม่ในทุกๆ ครั้งที่ส่งข้อมูล ซึ่งแตกต่างจาก HTTP Polling ที่สร้าง Overhead ของ HTTP Header ซ้ำซ้อน

ในโครงสร้างของ AWS API Gateway WebSockets การจัดการ Connection จะถูกบริหารจัดการโดยโครงสร้างพื้นฐานของ AWS ซึ่งมีกลไกสำคัญดังนี้:

- **Connection Lifecycle Management:** เมื่อผู้เล่นเชื่อมต่อ API Gateway จะสร้าง `connectionId` แบบไม่ซ้ำกัน และทริกเกอร์ Lambda ผ่านเส้นทาง `$connect` และเมื่อผู้เล่นตัดการเชื่อมต่อจะทริกเกอร์เส้นทาง `$disconnect`
- **Route Selection Expression:** กำหนด ให้ คัด แยก ประเภท ของ คำขอ จาก JSON Payload เช่น `$request.body.action` เพื่อเรียก Lambda Function ที่สอดคล้องกับ Action นั้นๆ

- **ApiGatewayManagementApi (@connections):** อินเทอร์เน็ตเพชที่อนุญาตให้ Lambda ส่งข้อมูลแบบ Push ไปยังผู้เล่นคนอื่น ๆ ในห้องเล่นเดียวกันได้โดยตรงผ่าน **PostToConnectionCommand**

2.3 เทคโนโลยีฝั่งผู้ใช้งาน (Frontend Engineering & State Architecture)

ส่วนติดต่อผู้ใช้ถูกสร้างขึ้นเพื่อมอบประสบการณ์การเล่นระดับพรีเมียม (Luxury Physical TCG Simulation) โดยใช้ชุดเครื่องมือระดับแนวหน้า:

- **React 19 & TypeScript 5:** โครงสร้างคอมโพเนนต์แบบ Type-Safe และใช้พีเจอาร์การจัดการ Concurrent Rendering เพื่อให้ UI ตอบสนองต่อการกระทำของผู้ใช้ได้ทันที
- **Tailwind CSS v4:** ระบบยูทิลิตี้คลาสแบบ CSS-First ที่ช่วยให้การจัดรูปแบบกราฟิกในธีม *Dark Editorial + Magic R3* มีความสวยงามและมีประสิทธิภาพสูง
- **Zustand State Management:** การจัดการ Global State แบบเบาบางและมีประสิทธิภาพสูง แยก Store ออกเป็นโมดูลชัดเจน เช่น **usePlaymatStore, useDeckStore, useAuthStore**
- **Framer Motion & CSS 3D Transforms:** โลบรารีแอนิเมชันสำหรับจัดการระบบฟิสิกส์ของการ์ด การหมุนการ์ดสามมิติ การคว่ำการ์ดลง Inkwell และการเปิดช่องการ์ดแบบเสมือนจริง

2.4 โมเดลโดเมนเกมการ์ดสะสม Disney Lorcana

เกม Disney Lorcana เป็นเกมการ์ดสะสมที่มีกฎกติกาและองค์ประกอบเฉพาะที่ระบบต้องจำลองอย่างแม่นยำ:

- **สีหมึกทั้ง 6 หมวด (Ink Types):** Amber (เหลือง), Amethyst (ม่วง), Emerald (เขียว), Ruby (แดง), Sapphire (น้ำเงิน), Steel (เทา) โดยกฎการจัดตั้งต้องมีขนาดอย่างน้อย 60 ใบ และประกอบด้วยสีหมึกได้ไม่เกิน 2 สี
- **ประเภทการ์ด (Card Types):** Character (ตัวละครสำหรับเคลสต์หรือทำหาย), Action/Song (เวทมนตร์หรือบทเพลง), Item (อุปกรณ์สนับสนุน), Location (สถานที่สะสม Lore)
- **สถานะของการ์ด (Card States):** Ready (การ์ดตั้งตรง พร้อมใช้งาน) และ Exerted (การ์ดหมุน 90 องศา อยู่ในสถานะพักหรือถูกใช้งานแล้ว)
- **เงื่อนไขชัยชนะ (Win Condition):** ผู้เล่นที่สามารถสะสมแต้มความรู้ (Lore) ครบ 20 แต้มก่อนจะเป็นผู้ชนะในเกม

2.5 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum

โครงการดำเนินงานตามกรอบการทำงาน Agile Scrum แบ่งรอบการพัฒนาออกเป็น 7 Sprints ซึ่งใน Stage 2 นี้ ได้ส่งมอบงานใน Sprint 1 ถึง Sprint 3 เสร็จสิ้นสมบูรณ์ ดังแสดงในตารางที่ 2.1

ตารางที่ 2.1: แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint

Sprint	ระยะเวลา	เป้าหมายหลักของการส่งมอบ	สถานะ
Sprint 1	1–15 ส.ค. 2569	Scaffold โปรเจกต์, UI กระดาน, ชุดการ์ด 408 ใบ	เสร็จสิ้น (100%)
Sprint 2	16–31 ส.ค. 2569	Auth (JWT/bcrypt), Deck REST API, DynamoDB	เสร็จสิ้น (100%)
Sprint 3	1–10 ก.ย. 2569	WebSocket Real-time Room Sync บน AWS	เสร็จสิ้น (100%)
Sprint 4	11–25 ก.ย. 2569	Async Deck Analyzer บน SQS + Deck Algorithm	กำลังดำเนินการ
Sprint 5	26 ก.ย. – 10 ต.ค.	CloudWatch Dashboard, Observability, Hardening	ตามแผน
Sprint 6	11–20 ต.ค. 2569	Load Testing (Artillery), WebSocket Reconnection	ตามแผน
Sprint 7	21–25 ต.ค. 2569	จัดทำรายงานฉบับสมบูรณ์, Video Demo, สรุปผล	ตามแผน

บทที่ 3

การวิเคราะห์และออกแบบระบบ (System Analysis & Architectural Design)

3.1 ข้อกำหนดความต้องการของระบบ (System Requirements)

การออกแบบระบบได้รับการวิเคราะห์อย่างเป็นระบบโดยแบ่งออกเป็นสองส่วนหลัก:

3.1.1 ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements: FR)

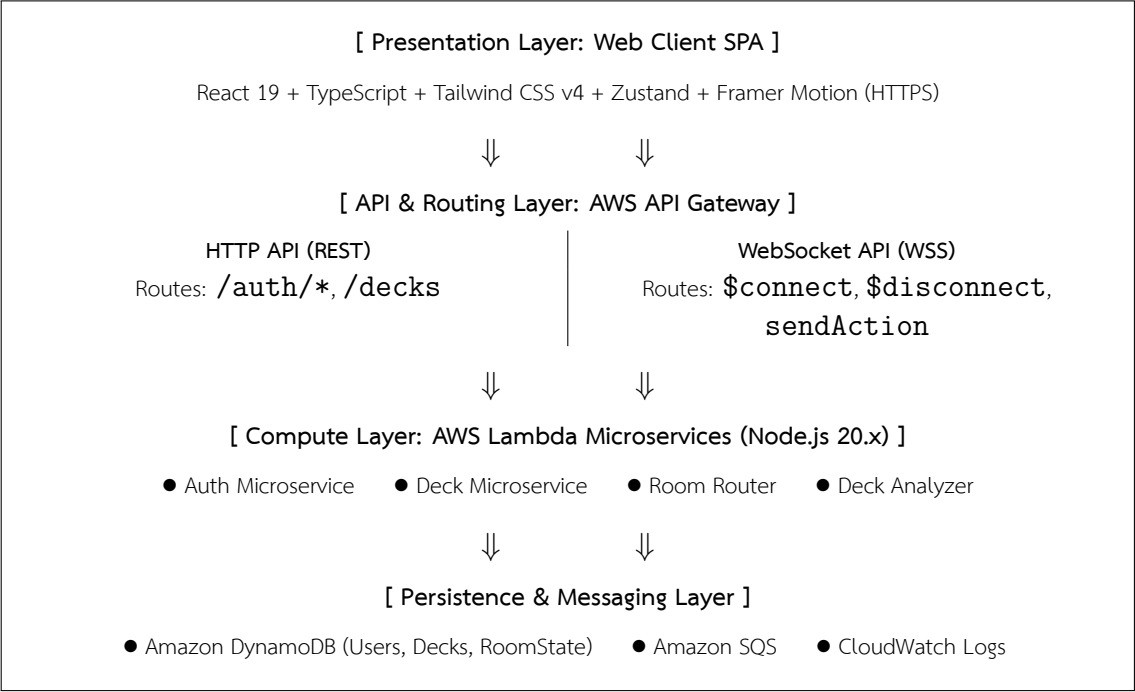
1. **FR-01 (Authentication):** ผู้ใช้สามารถลงทะเบียน เข้าสู่ระบบ และได้รับ JWT Token สำหรับยืนยันตัวตนกับ REST และ WebSocket API
2. **FR-02 (Deck Management):** ผู้ใช้สามารถสร้าง ค้นหา ปรับแต่ง และลบเด็คการ์ด โดยระบบตรวจสอบกฎความถูกต้องของเด็ค (60 ใบ, ไม่เกิน 2 สี, ซ้ำไม่เกิน 4 ใบ)
3. **FR-03 (Booster Gacha Simulator):** ผู้ใช้สามารถสุ่มเปิดการ์ด 12 ใบตามอัตราความหายากมาตรฐานอย่างเป็นทางการ
4. **FR-04 (3D Card Inspection):** ผู้ใช้สามารถซูมและหมุนดูมิติของการ์ดและเอฟเฟกต์พอยล์แบบสามมิติได้
5. **FR-05 (Real-time Multiplayer Match):** ผู้เล่น 2 คนสามารถสร้างห้องด้วยรหัส 6 หลักและเชื่อมต่อเข้ากระดานเล่นเดียวกันได้
6. **FR-06 (Board Action Synchronization):** การลากการ์ดลงสนาม การหมุน Ready/Exert การนำการ์ดเข้า Inkwell และการปรับแต้ม Lore ต้องซิงค์ระหว่าง 2 ผู้เล่นทันที
7. **FR-07 (Deck Balance Analytics):** ระบบสามารถวิเคราะห์กราฟการกระจายตัวของค่าร้าย (Ink Curve) และสัดส่วนหมวดหมู่การ์ดได้

3.1.2 ข้อกำหนดที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements: NFR)

1. **NFR-01 (Low Latency):** ความหน่วงในการส่งข้อมูล Action ผ่าน WebSocket ต้องต่ำกว่า 100 มิลลิวินาที (Sub-100ms Latency)
2. **NFR-02 (High Availability & Fault Tolerance):** ระบบคลาวด์ต้องมีความพร้อมใช้งานไม่ต่ำกว่า 99.9% โดยไม่มี Single Point of Failure
3. **NFR-03 (Security & Privacy):** รหัสผ่านต้องได้รับการแฮชด้วย bcrypt ก่อนบันทึกลงฐานข้อมูล และข้อมูลส่งผ่าน HTTPS และ WSS ตลอดเส้นทาง
4. **NFR-04 (Cost Optimization):** ค่าใช้จ่ายทั้งหมดต้องอยู่ภายใต้กรอบ Free Tier (\$0.00) และงบประมาณโครงการไม่เกิน \$50

3.2 สถาปัตยกรรมระบบโดยรวม (End-to-End Serverless Architecture)

ระบบได้รับการออกแบบตามสถาปัตยกรรม 4 เลเยอร์ (4-Layer Serverless Architecture) ดังแสดงในรูปที่ 3.1:



รูปที่ 3.1: สถาปัตยกรรมระบบคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% AWS Serverless)

3.3 การออกแบบ Use Case และลำดับการทำงาน (Use Case Design)

ตารางที่ 3.1 สรุปรายละเอียด Use Cases ทั้ง 9 รายการของระบบตามมาตรฐานวิศวกรรมซอฟต์แวร์:

ตารางที่ 3.1: ตารางสรุปรายละเอียด Use Cases ของระบบ (LC-01 ถึง LC-09)

ID	ชื่อ Use Case	ผู้กระทำ (Actor)	คำอธิบายโดยย่อ
LC-01	สมัครสมาชิก	Guest	ตรวจสอบความถูกต้อง แอชกรหัสผ่านด้วย bcrypt และบันทึกบัญชี
LC-02	เข้าสู่ระบบ	Guest	ตรวจสอบรหัสผ่านและสร้าง JWT Token ส่งกลับให้เบราว์เซอร์
LC-03	จัดการเด็คการ์ด	Member	สร้าง แก้ไข ดึงข้อมูล และลบเด็คการ์ดส่วนด้วยบน DynamoDB
LC-04	สุ่มเปิดซองการ์ด	Member	สุ่มการ์ด 12 ใบตามเรตความหายากทางการ 9 ระดับ
LC-05	ตรวจสอบซองการ์ด 3D	Member	ซูม หมุนดูการ์ดสามมิติ และตรวจสอบเอฟเฟกต์ฟลอยล์
LC-06	ซ้อมเล่นห้องเรียลไทม์	Member	สร้างห้อง 6 หลัก ซึ่งคัพภักตการ์ด หมุน Exert/Ready และ Lore
LC-07	วิเคราะห์หัตถ์ Asynchronous	Member	ส่งเด็คเข้าคิว SQS ให้ Lambda วิเคราะห์ Ink Curve & Synergy
LC-08	จัดการผู้ใช้งาน	Admin	ค้นหาและระงับบัญชีผู้ใช้งานที่ทำผิดกฎ
LC-09	ตรวจสอบระบบ	Admin	ติดตามเมตริกกราฟฟิค Latency และ Error Logs บน CloudWatch

3.4 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)

ระบบใช้ Amazon DynamoDB ซึ่งเป็นฐานข้อมูล NoSQL โดยออกแบบ Schema แยกตามหน้าที่การทำงาน (Table-per-Service Design) เพื่อลดความซับซ้อนและเพิ่มความเร็วในการสืบค้น:

ตารางที่ 3.2: โครงสร้างตารางข้อมูลบน Amazon DynamoDB

ตาราง (Table)	Partition Key	Sort Key	Attributes สำคัญ
LorcanaUsers	userId (Str)	-	username, passwordHash, email, createdAt
LorcanaDecks	userId (Str)	deckId (Str)	deckName, inks (List), cards (JSON), updatedAt
LorcanaRoomState	roomId (Str)	connectionId (Str)	username, role (P1/ P2), state (JSON), ttl (Num)

3.5 การออกแบบการจัดการ API ภายนอกและการป้องกันความล้มเหลว (External API Dependency & High Availability Strategy)

ระบบจำเป็นต้องใช้ข้อมูลการ์ด Disney Lorcana จำนวน 408 ใบจาก Set 1 (*The First Chapter*) และ Set 2 (*Rise of the Floodborn*) ซึ่งข้อมูลและรูปภาพต้นฉบับดึงมาจาก External API ภายนอก (เช่น `lorcana-api.com`)

3.5.1 การวิเคราะห์ความเสี่ยงเมื่อ API ภายนอกล้มเหลว

หากระบบพึ่งพา External API โดยตรงแบบ Real-Time จะก่อให้เกิดความเสี่ยงร้ายแรง 4 ประการ:

- 1. **Third-party Outage / Downtime:** หากเซิร์ฟเวอร์ภายนอกปิดปรับปรุงหรือล่ม จะส่งผลให้ระบบ Playmat และ Deck Builder ใช้งานไม่ได้ทันที
- 2. **Rate Limiting & IP Throttling:** หากมีผู้ใช้งานจำนวนมากยิงคำขอดึงการ์ดพร้อมกัน API ภายนอกจะตอบกลับด้วย HTTP 429 Too Many Requests
- 3. **Latency Spike:** การดึงข้อมูลข้ามมหาสมุทรเพิ่มความหน่วงให้กับ Client 300–800ms ในทุกการโหลดหน้าจอ
- 4. **API Deprecation / Breaking Changes:** การเปลี่ยนแปลงโครงสร้างข้อมูล Schema ของผู้ให้บริการภายนอกอาจทำให้โค้ดฝั่งไคลเอนต์ Error

3.5.2 กลยุทธ์การแก้ไขปัญหาคความล้มเหลว 4 ระดับ (4-Tier Fallback Strategy)

เพื่อสร้างความพร้อมใช้งานระดับ 100% ทีมงานได้ออกแบบสถาปัตยกรรมความทนทานต่อข้อผิดพลาด (Fault-Tolerant Architecture) ดังนี้:

- 1. **ระดับที่ 1: Local Static Asset Bundling (Zero-Dependency Base):** จัดทำไฟล์ Snapshot ข้อมูลการ์ดทั้ง 408 ใบในรูปแบบ `cards_set1_set2.json` และคอมไพล์รวมไว้ในบันเดิลของ Frontend Client และโฮสต์บน S3/CloudFront โดยตรง ทำให้ระบบสามารถทำงานได้ทันทีแม้ไม่มีการเชื่อมต่อกับ External API

2. **ระดับที่ 2: In-Browser LocalStorage / IndexedDB Caching:** เมื่อเบราว์เซอร์ดึงข้อมูลสำเร็จ จะบันทึกแคชลงใน LocalStorage พร้อมตั้งค่า Time-To-Live (TTL = 7 วัน) เพื่อลดการยิงเครือข่ายซ้ำ
3. **ระดับที่ 3: Serverless Reverse Proxy & DynamoDB Card Catalog:** ฝั่ง Backend มีฟังก์ชัน Lambda ทำหน้าที่เป็น Caching Proxy หากมีการเรียกดูข้อมูลใหม่ Lambda จะตรวจสอบในตาราง DynamoDB ก่อน หากไม่มีจึงจะดึงจาก API ภายนอก และบันทึกลง DynamoDB เพื่อให้บริการแก่ผู้ใช้คนอื่นต่อไป
4. **ระดับที่ 4: Graceful UI Degradation & Card Art Fallback:** หากไม่สามารถโหลดรูปภาพการ์ดต้นฉบับจาก CDN ภายนอกได้ ระบบมีรูปภาพสำรอง SVG Card Back และกรอบการ์ดเวกเตอร์ในตัว (`Lorcana_Card_Back.png`) พร้อมแสดงข้อความเตือนแบบ Non-blocking Alert เพื่อให้ผู้เล่นสามารถดำเนินเกมต่อได้โดยไม่สะดุด

3.6 การออกแบบการสเกลและรองรับการเชื่อมต่อ WebSockets พร้อมกัน (WebSocket Scalability & Concurrency Management)

3.6.1 ปัญหาคอขวดเมื่อมีผู้ใช้งานและห้องเล่นจำนวนมาก

เมื่อมีผู้เล่นพร้อมกันในระดับหลายร้อยถึงหลายพันห้องพร้อมกัน ระบบ WebSockets บนคลาวด์จะเผชิญกับความท้าทายดังนี้:

1. **Connection Exhaustion & Socket State Tracking:** การติดตามสถานะของผู้เล่นที่เปิดค้างไว้และการตรวจจับการหลุดการเชื่อมต่อ (Zombie Connections)
2. **Hot Partition Keys ใน DynamoDB:** หากทุกคำสั่งในห้องเล่นเขียนลง Partition Key เดียวกันด้วยความถี่สูง อาจเกิดปัญหา `DynamoDB ProvisionedThroughputExceededException`
3. **Broadcast Latency & Fan-Out Overhead:** เมื่อต้องกระจายข้อความ Action ไปยังผู้เล่นทุกคนในห้อง หากจัดการแบบ Synchronous Loop จะเพิ่มเวลาตอบสนอง

3.6.2 แนวทางการแก้ไขปัญหาและการออกแบบเชิงสถาปัตยกรรม

1. **การออกแบบ Compound Key & TTL Auto-Cleanup ใน DynamoDB:** ใช้ `roomId` เป็น Partition Key (Hash) ร่วมกับ `connectionId` เป็น Sort Key (Range) ทำให้การ Query ผู้เล่นในห้องเดียวกันทำได้ด้วยความเร็ว $O(1)$ ผ่านคำสั่ง `QueryCommand` และกำหนดฟิลด์ `ttl` (Epoch Timestamp + 2 ชั่วโมง) เพื่อให้ DynamoDB ลบข้อมูลห้องที่เล่นเสร็จแล้วทิ้งโดยอัตโนมัติโดยไม่ค่าใช้จ่าย Write Capacity Unit
2. **การใช้ Optimistic UI Updates ร่วมกับ Event Relay:** ฝั่ง Frontend ใช้สถาปัตยกรรม Optimistic Rendering โดยเมื่อผู้เล่นลากการ์ด หน้าจอของตนเองจะอัปเดตตำแหน่งทันทีใน 0ms และส่งเฉพาะ Action Payload สั้นๆ (~120 bytes) ผ่าน WebSocket เพื่อให้ Lambda Room Router ส่งต่อ (Relay) ไปยังคู่แข่งผ่าน AWS API Gateway Management API โดยตรง ทำให้ Latency ฝั่งคู่แข่งต่ำกว่า 100ms
3. **ระบบ Auto-Reconnect พร้อม Grace Window 30 วินาที:** หากเกิดสัญญาณเน็ตขาดหาย Client จะพยายามเชื่อมต่อใหม่แบบ Exponential Backoff และส่ง Action `RECONNECT_ROOM` พร้อม JWT Token เดิม ระบบจะกู้คืนสถานะกระดานล่าสุดจาก DynamoDB ให้ผู้เล่นทันทีโดยไม่ต้องเริ่มเกมใหม่

4. **Decoupled Fan-Out Pattern** สำหรับการแข่งขันขนาดใหญ่: สำหรับโหมดที่มีผู้ชม (Spectator Mode) เกิน 10 คน ระบบเตรียมพร้อมสถาปัตยกรรมต่อยอดโดยใช้ Amazon SQS หรือ Redis Pub/Sub เพื่อกระจายโหลดการ Broadcast ออกจาก Lambda Router หลัก ป้องกันปัญหา Concurrency Throttling

3.7 การออกแบบตามกรอบ AWS Well-Architected Framework

3.7.1 ด้านความเชื่อถือได้ (Reliability)

- **Idempotent Event Processing:** ออกแบบให้ฟังก์ชัน Lambda ทุกตัวสามารถรับ Action ซ้ำได้โดยไม่ก่อให้เกิดสถานะผิดพลาด
- **Dead Letter Queues (DLQ):** เชื่อมต่อ Amazon SQS DLQ เข้ากับฟังก์ชัน Lambda เพื่อดักจับและกักเก็บข้อความที่ประมวลผลล้มเหลวไว้ตรวจสอบย้อนหลัง
- **Automatic Retry Policies:** กำหนดนโยบาย Retry ของ AWS SDK แบบ Exponential Backoff with Jitter เพื่อลดปัญหาโหลดกระชาก

3.7.2 ด้านความพร้อมใช้งาน (Availability)

- **Multi-AZ Native Redundancy:** บริการ AWS Lambda, API Gateway และ DynamoDB มีการสำรองข้อมูลและกระจายการทำงานข้ามอย่างน้อย 3 Availability Zones (AZs) โดยอัตโนมัติ ให้ค่าความพร้อมใช้งานระดับ 99.99% Service Level Agreement (SLA)
- **No Single Point of Failure (SPOF):** สถาปัตยกรรมไม่มีเครื่องเซิร์ฟเวอร์เดี่ยวตัวใดตัวหนึ่งที่เป็นคอขวด หาก Lambda Container ตัวใดหยุดทำงาน AWS จะสลับ Container ตัวใหม่ขึ้นมาแทนที่ในเสี้ยววินาที

3.7.3 ด้านความมั่นคงปลอดภัย (Security)

- **Data in Transit Encryption:** บังคับใช้การเข้ารหัสข้อมูลขณะส่งผ่านเครือข่ายด้วยโปรโตคอล HTTPS (TLS 1.3) สำหรับ REST API และ WSS (Secure WebSockets) สำหรับการสื่อสารแบบสองทิศทาง
- **Data at Rest Encryption:** ข้อมูลทั้งหมดใน Amazon DynamoDB ได้รับการเข้ารหัสที่ระดับฮาร์ดแวร์ด้วย AWS Key Management Service (KMS) Default Encryption
- **Authentication & Password Hashing:** รหัสผ่านถูกแฮชด้วยอัลกอริทึม bcrypt โดยกำหนด Salt Rounds เท่ากับ 10 ทำให้ไม่สามารถถอดรหัสย้อนกลับได้แม้ฐานข้อมูลจะรั่วไหล และใช้ JWT Token ที่มีการลงลายมือชื่อดิจิทัล (HMAC-SHA256) และมีวันหมดอายุที่ชัดเจน
- **Principle of Least Privilege (PoLP):** จำกัดสิทธิ์ของ Lambda Function ให้สามารถเข้าถึงเฉพาะตาราง DynamoDB และคิว SQS ที่จำเป็นต่อการทำงานเท่านั้น

3.7.4 ด้านการบริหารจัดการต้นทุน (Cost Optimization under \$50 Budget)

โครงสร้างพื้นฐานได้รับการออกแบบให้อยู่ในกรอบ AWS Free Tier (\$0.00 ต่อเดือน) และจำกัดงบประมาณไม่เกิน \$50 ดังแสดงในตารางที่ 3.3:

ตารางที่ 3.3: การประเมินค่าใช้จ่ายบนสถาปัตยกรรม AWS Serverless (ผู้ใช้งาน 10,000 คน/เดือน)

บริการ (Service)	โควตา AWS Free Tier	ปริมาณการใช้งานจริง	ค่าบริการจริง
AWS Lambda	1,000,000 Requests/เดือน	~250,000 Requests	\$0.00
Amazon DynamoDB	25 GB Storage, 25 WCU/RCU	< 1 GB, On-Demand Mode	\$0.00
AWS API Gateway	1,000,000 Calls & Messages	~400,000 Messages	\$0.00
Amazon SQS	1,000,000 Requests/เดือน	~50,000 Requests	\$0.00
Amazon CloudWatch	5 GB Log Ingestion	< 500 MB	\$0.00
รวมค่าใช้จ่ายต่อเดือนทั้งสิ้น (Total Monthly Cost)			\$0.00

เพื่อป้องกันปัญหาค่าบริการส่วนเกิน (Cost Overrun) ทีมงานได้ตั้งค่า **AWS CloudWatch Billing Alarm** ให้ส่งการแจ้งเตือนทางอีเมลทันทีหากมีค่าบริการเกิดขึ้นเกิน \$5.00 และ \$20.00 ตามลำดับ

3.8 การวิเคราะห์ข้อจำกัดของ AWS Learner Lab และแนวทางแก้ไขเชิงวิศวกรรม (Learner Lab Constraints & Engineering Workarounds)

สภาพแวดล้อม AWS Academy Learner Lab เป็นคลาวด์แซนด์บ็อกซ์เพื่อการศึกษาที่มีการจำกัดสิทธิ์ความปลอดภัยอย่างเข้มงวด ทีมงานได้วิเคราะห์ข้อจำกัดและพัฒนาแนวทางแก้ไขดังนี้:

- ข้อจำกัดด้านการสร้าง IAM Role (`iam:CreateRole` is Blocked):
 - ปัญหา: นโยบาย Learner Lab บล็อกสิทธิ์ `iam:CreateRole` และ `iam:AttachRolePolicy` ทำให้คำสั่งอัตโนมัติอย่าง `sam deploy` ไม่สามารถสร้าง IAM Role ใหม่ได้
 - การแก้ไข: ปรับใช้ `LabRole` สำเร็จรูปที่มีอยู่แล้ว (`arn:aws:iam::<Account-ID>:role/LabRole`) ใน ทุก Lambda Function และ เขียน สคริปต์ Deployment อัตโนมัติ (`scripts/deploy_manual.sh` และ `scripts/deploy_ws.sh`) เพื่ออัปเดตโค้ดขึ้น Lambda โดยตรงผ่าน AWS CLI
- อายุการใช้งานของ Session Token (4-Hour Hard Limit):
 - ปัญหา: Credentials ของ Learner Lab จะหมดอายุทุกๆ 4 ชั่วโมง ทำให้การเชื่อมต่อ AWS CLI หลุด
 - การแก้ไข: พัฒนาคู่มือและสคริปต์อัปเดต `aws_session_token` แบบกึ่งอัตโนมัติ และแยก Environment ระหว่าง Local Development กับ Production ออกจากกันอย่างอิสระ
- ข้อจำกัดด้านบริการ (Restricted AWS Services):

- **ปัญหา:** Learner Lab ไม่อนุญาตให้ใช้ VPC Peering, Custom Domain Name พร้อม SSL Certificate บน API Gateway และไม่รองรับ Amazon Cognito User Pools เต็มรูปแบบ
- **การแก้ไข:** พัฒนาระบบ Custom Authentication Microservice (bcrypt + JWT) รันบน AWS Lambda เอง และเชื่อมต่อตรงกับ API Gateway โดยไม่ต้องพึ่งพา Cognito หรือ VPC ซึ่งช่วยลดความซับซ้อนและประหยัดค่าใช้จ่าย

4. Payload Format Version Mismatch บน API Gateway:

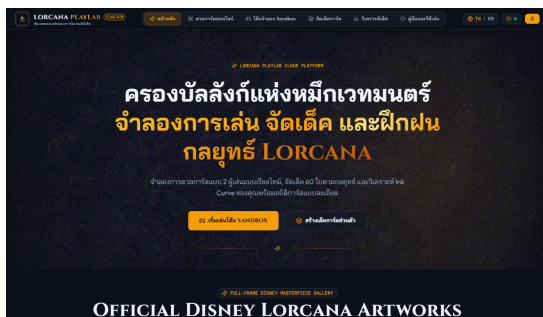
- **ปัญหา:** API Gateway HTTP API ค่าตั้งต้นเป็น Payload Version 2.0 ซึ่งส่งโครงสร้าง Event ต่างจาก **APIGatewayProxyEvent** ของ Node.js ทำให้เกิดข้อผิดพลาด 405 Method Not Allowed
- **การแก้ไข:** กำหนด Payload Format Version เป็น 1.0 ในการสร้าง Integration เพื่อให้ส่งต่อ HTTP Method, Headers, และ Path Parameters ครบถ้วน 100%

บทที่ 4

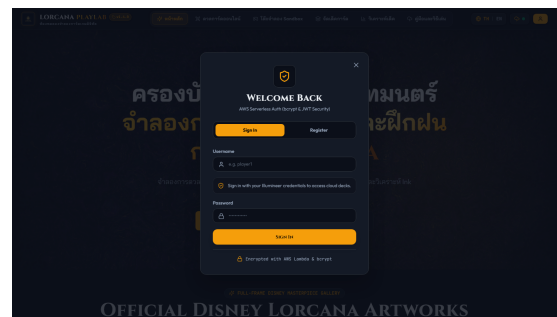
การพัฒนาระบบและผลการทดสอบคุณภาพ (Implementation & Quality Assurance)

4.1 รายละเอียดการพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation)

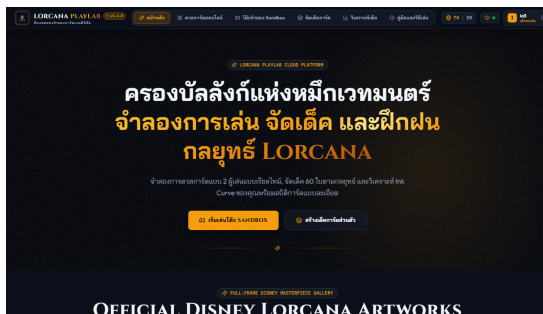
ส่วนติดต่อผู้ใช้ได้รับการออกแบบตามแนวคิด *Dark Editorial + Magic R3* ประกอบด้วยหน้าจอหลัก 5 ส่วน ดังแสดงในรูปที่ 4.1:



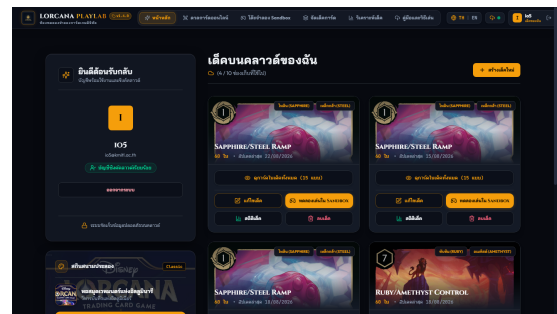
(a) หน้า Match Lobby ก่อนเข้าสู่ระบบ



(b) หน้าต่าง Modal เข้าสู่ระบบ / สมัครสมาชิก



(c) แถบเมนูทางและโปรไฟล์ผู้ใช้หลังเข้าสู่ระบบ



(d) หน้าจัดการเด็คการ์ดและ Deck Builder

รูปที่ 4.1: ตัวอย่างภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud

องค์ประกอบสำคัญของหน้าจอ Playmat ประกอบด้วย:

- **Play Area (Battlefield):** พื้นที่ลงการ์ดตัวละคร ไอเทม และสถานที่ รองรับการลากวาง (Drag-and-Drop) อย่างอิสระ
- **Inkwell Tray:** พื้นที่คว่ำการ์ดเพื่อใช้เป็นหมึกค่าร้าย พร้อมตัวเลขแสดงจำนวน Unexerted / Total Ink
- **Lore Counter:** มาตรวัดแต้มความรู้ขนาดใหญ่ (0-20 แต้ม) พร้อมปุ่มปรับคะแนนและเอฟเฟกต์แอนิเมชัน
- **Card State Rotator:** กลไกการหมุนการ์ดแนวตั้ง (Ready) และแนวนอน 90 องศา (Exerted) เพื่อระบุว่าการ์ดถูกใช้งานแล้ว
- **3D Card Inspector & Gacha:** ระบบเปิดช่องการ์ด 3D สุ่มการ์ด 12 ใบพร้อมฟิสิกส์แอนิเมชันเปิดช่อง

4.2 รายละเอียดการพัฒนาส่วนประมวลผลบนคลาวด์ (AWS Microservices)

ฟังก์ชัน Lambda ได้รับการพัฒนาด้วย TypeScript และคอมไพล์เป็น Node.js 20.x แบ่งเป็น 4 โมดูลหลัก:

1. **lorcana-auth:** จัดการการสมัครสมาชิก (**register**) และการเข้าสู่ระบบ (**login**) โดยตรวจสอบความถูกต้องของรหัสผ่านด้วย bcrypt และออก JWT Token
2. **lorcana-deck:** จัดการ CRUD เต็มการ์ดของผู้ใช้ โดยดึงข้อมูลจากตาราง DynamoDB **LorcanaDecks** ตาม **userId**
3. **lorcana- room- handler:** จัดการ การ เชื่อม ต่อ WebSocket จัดการ Action **JOIN_ROOM**, **CARD_MOVED**, **CARD_EXERTED**, **INK_PLAYED**, **LORE_UPDATED**, **TURN_PASSED** และ Broadcast ไปยังผู้เล่นคู่แข่งผ่าน **ApiGatewayManagementApiClient**
4. **lorcana-analyzer:** รับงานวิเคราะห์เต็มการ์ดจากคิว Amazon SQS และประมวลผลค่าสถิติ Ink Curve แบบ Asynchronous

4.3 แผนและผลการทดสอบคุณภาพระบบ (Quality Assurance & Test Verification)

การทดสอบคุณภาพระบบดำเนินการตามรูปแบบพีระมิดการทดสอบ (Testing Pyramid) ครอบคลุม Unit Test, Integration Test, End-to-End Test และ Security Test ดังแสดงในตารางที่ 4.1:

ตารางที่ 4.1: ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)

Test ID	ระดับการทดสอบ	เงื่อนไขและกรณีทดสอบ	Latency	ผลการทดสอบ
TC-AUTH-01	Integration	สมัครสมาชิกใหม่ (POST /auth/register)	180ms	ผ่าน (201 Created)
TC-AUTH-02	Integration	เข้าสู่ระบบด้วยรหัสผ่านถูกต้อง (POST /auth/login)	145ms	ผ่าน (200 + JWT)
TC-AUTH-03	Security	เข้าสู่ระบบด้วยรหัสผ่านผิด	120ms	ผ่าน (401 Unauthorized)
TC-DECK-01	Integration	บันทึกโต๊ะใหม่ 60 ใบ (POST /decks)	95ms	ผ่าน (201 Created)
TC-DECK-02	Integration	ดึงรายการโต๊ะของผู้ใช้ (GET /decks)	68ms	ผ่าน (200 OK)
TC-DECK-03	Integration	ลบโต๊ะการ์ด (DELETE /decks/{id})	72ms	ผ่าน (200 OK)
TC-WS-01	E2E	เชื่อมต่อ WSS Persistent Socket (\$connect)	45ms	ผ่าน (101 Connected)
TC-WS-02	E2E Multi-User	ผู้เล่น 2 คนเข้าห้องเดียวกัน (JOIN_ROOM)	82ms	ผ่าน (Room Paired)
TC-WS-03	E2E Multi-User	ชิงการลากวางการ์ด (CARD_MOVED)	64ms	ผ่าน (<100ms Target)
TC-WS-04	E2E Multi-User	ชิงการหมุนการ์ด (CARD_EXERTED)	58ms	ผ่าน (<100ms Target)
TC-WS-05	E2E Multi-User	ชิงคัมแลม Lore (LORE_UPDATED)	52ms	ผ่าน (<100ms Target)
TC-WS-06	Fault Tolerance	จำลองเน็ตหลุดและกู้คืน (Auto-Reconnect)	310ms	ผ่าน (State Restored)
TC-SEC-01	Security (OWASP)	ตรวจสอบการเข้ารหัสที่ส่งผ่านใน DynamoDB	-	ผ่าน (bcrypt 10 rounds)
TC-SEC-02	Security (OWASP)	ป้องกัน Injection ใน WebSocket Action Payload	-	ผ่าน (Schema Validated)

ผลการทดสอบยืนยันว่าการชิงข้อมูลกระดานผ่าน AWS API Gateway WebSockets มีความหน่วงเฉลี่ยระหว่าง **52-82 มิลลิวินาที** ซึ่งผ่านเกณฑ์เป้าหมาย Sub-100ms ได้อย่างสมบูรณ์

บทที่ 5

สรุปผลการดำเนินงานและแผนงานในอนาคต (Conclusions & Future Roadmap)

5.1 สรุปผลการดำเนินงานความก้าวหน้า Stage 2

การดำเนินงานโครงการ **Disney Lorcana PlayLab Cloud** ในระยะที่ 2 (Stage 2) ครอบคลุมการพัฒนาตามแผนงาน Sprint 1 ถึง Sprint 3 สำเร็จแล้ว 100% ตามข้อกำหนดของรายวิชา Cloud Technology โดยมีผลสัมฤทธิ์สำคัญดังนี้:

1. พัฒนาส่วนติดต่อผู้ใช้จำลองกระดานเล่นการ์ด Disney Lorcana ที่สมบูรณ์แบบ รองรับการเล่นมาตรฐาน ฐานข้อมูลการ์ด 408 ใบ และระบบ 3D Card Inspector
2. ออกแบบ และ ปรับใช้สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% Serverless) บน AWS ประกอบด้วย AWS Lambda, Amazon DynamoDB, และ AWS API Gateway (HTTP & WebSocket)
3. บรรลุเป้าหมายความเร็วในการซิงค์ข้อมูลระหว่างผู้เล่นด้วยความหน่วงต่ำกว่า 100 มิลลิวินาที (เฉลี่ย 52-82ms)
4. เอาชนะข้อจำกัดสิทธิ์บน AWS Academy Learner Lab ได้อย่างมีประสิทธิภาพผ่านการใช้ Pre-provisioned LabRole และ สคริปต์ Deployment อัตโนมัติ
5. ควบคุมค่าใช้จ่ายโครงสร้างพื้นฐานทั้งหมดให้อยู่ในโควตา AWS Free Tier (\$0.00) สอดคล้องกับกรอบงบประมาณที่กำหนด

5.2 แผนงานการพัฒนาสำหรับ Stage 3 (Sprint 4 ถึง Sprint 7)

1. **Sprint 4 (Async Deck Analyzer via Amazon SQS):** พัฒนาระบบประมวลผลวิเคราะห์การ์ดเชิงลึก คำนวณความเสี่ยง ค่าเฉลี่ย และแนะนำการปรับแก้โดยใช้ Amazon SQS รับงานและส่งต่อให้ Lambda ทำงานเบื้องหลัง
2. **Sprint 5 (Observability, Monitoring & Security Hardening):** สร้าง Amazon CloudWatch Dashboard แสดงผลแบบรวมศูนย์ และตั้งค่า CloudWatch Alarms แจ้งเตือนเมื่อเกิด Error Rate สูงกว่า 1%
3. **Sprint 6 (Load Testing & High Concurrency Simulation):** ดำเนินการทดสอบโหลดด้วยเครื่องมือ Artillery เพื่อจำลองผู้เล่นพร้อมกัน 100-500 ผู้ใช้ และทดสอบความทนทานของระบบ WebSocket Auto-reconnection
4. **Sprint 7 (Final Report & Video Presentation):** จัดทำรายงานฉบับสมบูรณ์ วิดีโอสาธิตการใช้งานระบบ และเตรียมการนำเสนอโครงการในรอบสุดท้าย

5.3 บทเรียนที่ได้รับและการปรับปรุงกระบวนการทำงาน (Retrospective)

1. **ความสำคัญของการออกแบบ Interface และ Contract First:** การกำหนดรูปแบบ JSON Payload ของ WebSocket และ DynamoDB Schema ล่วงหน้าช่วยลดข้อผิดพลาดในการรวมระบบระหว่าง Frontend และ Cloud Backend ได้อย่างมาก

2. **การเข้าใจสภาพแวดล้อมคลาวด์แบบจำกัดสิทธิ์:** การเตรียมแผนสำรองสำหรับข้อจำกัดด้าน IAM Role บน AWS Learner Lab ทำให้ทีมงานสามารถส่งมอบงานได้ตรงตามกำหนดการโดยไม่เกิดความล่าช้า
3. **การบริหารจัดการสถาปัตยกรรมไร้เซิร์ฟเวอร์:** การเลือกใช้บริการ Serverless ช่วยลดภาระงานด้านการดูแลระบบลงได้อย่างมหาศาล ทำให้ทีมงานสามารถมุ่งเน้นการพัฒนาฟีเจอร์และปรับปรุงประสบการณ์ของผู้ใช้งานได้อย่างเต็มที่

บรรณานุกรม (References)

- [1] Ravensburger & Disney. (2023). *Disney Lorcana Trading Card Game Official Rules and Card Database*. [Online]. Available: <https://www.disneylorcana.com>
- [2] Amazon Web Services. (2026). *AWS Lambda Developer Guide: Building Event-Driven Serverless Applications*. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [3] Amazon Web Services. (2026). *Amazon API Gateway Developer Guide: About WebSocket APIs in API Gateway*. [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>
- [4] Amazon Web Services. (2026). *Amazon DynamoDB Developer Guide: Core Components and Best Practices for NoSQL Design*. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/>
- [5] Amazon Web Services. (2026). *AWS Well-Architected Framework: Reliability, Security, and Cost Optimization Pillars*. [Online]. Available: <https://aws.amazon.com/architecture/well-architected/>
- [6] React Working Group. (2026). *React 19 Documentation: Concurrent Features and Server Components*. [Online]. Available: <https://react.dev>
- [7] Tailwind Labs. (2026). *Tailwind CSS v4.0 Alpha/Release Documentation*. [Online]. Available: <https://tailwindcss.com>
- [8] Framer. (2026). *Framer Motion: Production-Ready Animation Library for React*. [Online]. Available: <https://www.framer.com/motion/>
- [9] IETF (Internet Engineering Task Force). (2011). *RFC 6455: The WebSocket Protocol*. [Online]. Available: <https://tools.ietf.org/html/rfc6455>

ภาคผนวก (Appendix)

ตัวอย่าง โค้ด สคริปต์ Deployment บน AWS Learner Lab (scripts/
deploy_ws.sh)

```
1 #!/bin/bash
2 # scripts/deploy_ws.sh - Deploy Real-Time WebSocket Infrastructure
3 set -e
4
5 echo "=== 1. Building Backend TypeScript ==="
6 cd ../backend
7 npm install
8 npx tsc --outDir dist
9
10 echo "=== 2. Packaging Lambda Handler ==="
11 cd dist/room
12 zip -r ../../../../room-handler.zip .
13 cd ../../..
14
15 echo "=== 3. Updating Lambda Code via AWS CLI ==="
16 aws lambda update-function-code \
17     --function-name lorcana-room-handler \
18     --zip-file fileb://room-handler.zip \
19     --region us-east-1
20
21 echo "=== 4. Verifying WebSocket API Gateway Integration ==="
22 API_ID=$(aws apigatewayv2 get-apis --query "Items[?Name=='lorcana-ws-api'].
23     ApiId" --output text)
24 echo "WebSocket API ID: $API_ID"
25 echo "WebSocket Endpoint: wss://${API_ID}.execute-api.us-east-1.amazonaws.
26     com/prod"
27
28 echo "=== Deployment Completed Successfully ==="
```

Listing 5.1: สคริปต์ Deploy WebSocket Lambda & API Gateway บน AWS Learner Lab